



Final Project - Data Integrity and Authentication

Secure Document Vault System

In this project, each team is required to design and implement a secure web-based platform called Secure Document Vault. The system will allow users to securely upload, store, manage, encrypt, sign, and verify digital documents while applying modern concepts of authentication, authorization, encryption, integrity verification, and secure communication.

The project aims to simulate a real-world secure enterprise system that protects sensitive documents against unauthorized access, tampering, and interception. Students are expected to integrate multiple security mechanisms into one complete application and demonstrate how modern systems achieve confidentiality, integrity, authenticity, and secure access control.

The system must contain a complete web-based user interface. Using Postman alone is not acceptable.

Project Objectives

The main objectives of this project are:

- Understand and implement modern authentication mechanisms.
- Apply secure password storage and password policy validation.
- Implement Role-Based Access Control (RBAC).
- Secure APIs and protected resources using JWT authentication.
- Integrate OAuth login using GitHub or Google.
- Implement Two-Factor Authentication (2FA).
- Apply encryption techniques to protect uploaded documents.
- Implement digital signatures and integrity verification.
- Secure communication using HTTPS.
- Demonstrate the importance of HTTPS through a simulated MITM traffic analysis using Wireshark.
- Build a complete secure web application similar to real-world enterprise systems.

Functional Requirements

The system should include the following modules and features.

1. Authentication and User Management

The platform must support secure user authentication and account management.

Required features:

- User registration and login
- Password hashing using secure algorithms such as bcrypt or Argon2
- Password policy enforcement



- JWT-based authentication
- OAuth login using GitHub or Google
- Two-Factor Authentication (2FA)
- Logout functionality
- Passwords must never be stored in plain text.

2. Role-Based Access Control (RBAC)

The system must implement multiple user roles with different permissions.

Roles:

- Admin
- Manager
- User

Permissions:

- Admin can manage users and roles.
- Managers can review and verify documents.
- Users can upload and manage their own documents.

3. Secure Document Management

Users should be able to upload and manage documents securely through the web interface.

The system should support:

- Uploading documents
- Viewing uploaded documents
- Downloading documents
- Deleting documents
- Viewing document metadata

You must validate uploaded file types and file sizes.

4. Document Encryption

Uploaded documents must be encrypted before being stored on the server. The encryption process should ensure that documents cannot be read directly from storage.

Students may use algorithms such as AES-256 for encryption.

5. Digital Signatures and Integrity Verification

The system must support digital signatures and document integrity verification.

For every uploaded document:



- Generate a SHA-256 hash
- Digitally sign the document or its hash
- Store the generated signature

The system should also allow users to verify whether a document has been modified after upload.

6. HTTPS and Secure Communication

The application must run using HTTPS.

You should configure HTTPS certificates and demonstrate secure communication between the client and the server.

7. MITM Simulation Using Wireshark

You are required to demonstrate the importance of HTTPS using Wireshark.

The demonstration should include:

- Capturing traffic while using HTTP
- Showing how sensitive information may appear in plain text
- Capturing traffic again after enabling HTTPS
- Explaining how HTTPS protects transmitted data

Technologies

There are no restrictions on the technologies or frameworks used in this project. Teams may choose any suitable backend, frontend, database, or UI technologies.

However, the project must properly implement the required security concepts including:

- JWT Authentication
- Password Hashing
- Password Policy Enforcement
- OAuth Authentication
- Role-Based Access Control (RBAC)
- Two-Factor Authentication (2FA)
- Document Encryption
- Digital Signatures
- Data Integrity Verification
- HTTPS Secure Communication
- MITM Traffic Analysis using Wireshark

Teams are required to use well-known, trusted, and well-vetted libraries/frameworks for implementing security features. Students may use any suitable libraries, frameworks, or tools that follow secure and industry-standard practices.



Team Requirements

1. Each team should consist of 3–5 students.
2. All team members are expected to actively participate in the entire project development process.
3. It is not allowed to divide the project into isolated responsibilities such as assigning one student only for HTML/UI, another only for database design, and another only for security implementation.
4. Every team member must contribute to and understand all major aspects of the system including:
 - Authentication and authorization
 - Database design
 - Frontend development
 - Backend implementation
 - Security mechanisms
 - Encryption and integrity verification
 - HTTPS configuration
 - Testing and debugging
5. During project discussions, demonstrations, and evaluation sessions, all team members are expected to be able to explain and discuss the implemented features and security concepts.
6. Teams are encouraged to collaborate continuously and review each other's work to ensure full understanding of the complete system architecture and implementation.
7. Any team where participation is clearly unbalanced may receive individual evaluation penalties during the final assessment.

User Interface Requirement

The system must contain a complete and functional web-based UI. Suggested pages include:

- Login page
- Registration page
- Dashboard
- Document upload page
- Document verification page
- Admin panel

Deliverables

- Full source code
- GitHub repository link
- Database script
- Project documentation/report
- Screenshots of the system
- Screenshots of Wireshark demonstrations
- Instructions explaining how to run the project in README file



Important Notes

- Postman-only projects are not accepted.
- Teams are encouraged to follow secure coding practices.
- Any plagiarism or copied projects will receive zero marks.

Submission and Demonstration

- Project submission and demonstrations will take place during Week 14 and Week 15.
- Each team must reserve an available presentation/demo time slot through the provided scheduling link.
- Teams are responsible for selecting their preferred available slot before the deadline.
- All team members are expected to attend the demonstration and participate in the discussion and explanation of the implemented features and security mechanisms.
- Any team member who does not attend the project demonstration and discussion session will receive **ZERO** for the project evaluation unless an official accepted excuse is provided.